Academic Emacs setup for LaTeX

Moinul Islam

2026-08-08

Table of contents

Preface

Emacs Configuration (MacOSX, LinuxOS)

This configuration is designed as a lightweight but powerful academic computing environment built on top of Emacs. It is intended to support a complete research workflow that spans note-taking, reproducible analysis, manuscript writing, and publication preparation within a single extensible system.

The core idea behind this setup is integration rather than fragmentation. Instead of relying on separate tools for reading, writing, coding, and referencing, this configuration brings everything into a unified environment where ideas can move seamlessly from raw notes to structured knowledge to publishable outputs.

The system is particularly optimized for academic research work, including LaTeX-based manuscript preparation (via AUCTeX and latexmk), interactive PDF navigation (via PDF Tools with SyncTeX) and literate knowledge management (via Org mode and Org-roam).

Overall, this configuration is not just an editor setup but a research operating environment. It is designed to scale from small notes to full-length academic manuscripts and to remain stable, reproducible, and maintainable over long-term research projects.

1 early-init.el

- Place early-init.el in ~/.emacs.d/

```
;;; early-init.el --- Early initialization

;;; Commentary:
;; This file loads before init.el, before package.el runs its own
;; automatic initialization, and before the frame is created. That
;; makes it the right place for:
;;   - GC tuning during startup (earlier = more of startup benefits)
;;   - package-system setup (package-enable-at-startup has no effect
;;     if set later, since Emacs has already acted on it by the time
;;     init.el loads)
;;   - anything that affects the very first frame, to avoid a visible
;;     flash of unstyled UI before init-ui.el's theme applies
;;
;; init.el restores gc-cons-threshold to a sane runtime value once
;; startup finishes (see its `emacs-startup-hook').

;;; Code:

;; ==================================================
;; GC tuning during startup
;; ==================================================
;; Very high threshold while package.el/use-package/all init-*.el
;; modules load, so frequent collections don't slow that process
;; down. This is only safe because init.el lowers it again once
;; `emacs-startup-hook' runs.
(setq gc-cons-threshold most-positive-fixnum
      gc-cons-percentage 0.6)

;; ==================================================
;; Package system
;; ==================================================
;; init.el calls `package-initialize' itself (after configuring
;; `package-archives'), so this prevents Emacs's own automatic
```

```
;; initialization from doing it a second time before init.el gets a
;; chance to set up the archive list.
(setq package-enable-at-startup nil)

;; ==================================================
;; Native compilation
;; ==================================================
;; Suppress noisy async native-comp warnings as early as possible
;; (some can otherwise appear before init.el even loads).
(setq native-comp-async-report-warnings-errors nil)

;; ==================================================
;; First-frame behavior
;; ==================================================
;; Avoid Emacs doing extra work to fit the frame to content during
;; the resize dance at startup -- faster, and avoids visible jitter.
(setq frame-inhibit-implied-resize t)

;; If you ever want to disable the menu bar / tool bar / scroll bar
;; (currently left commented out, opt-in, in init-system.el), doing
;; it here instead avoids a visible flash of them appearing and then
;; disappearing after the frame is already shown:
;;   (menu-bar-mode -1)
;;   (tool-bar-mode -1)
;;   (scroll-bar-mode -1)

(provide 'early-init)

;;; early-init.el ends here
```

2 init.el

- Keep init.el in ~/.emacs.d/

```
;;; init.el --- Academic Emacs Configuration

;; ====================================================
;; Performance (runtime, post-startup)
;; ====================================================
;; early-init.el set gc-cons-threshold very high for the duration of
;; startup (package loading etc.); bring it back down to a sane
;; runtime value once that's done, so GC still runs at a reasonable
;; cadence during normal editing.

(setq read-process-output-max (* 1024 1024))

(add-hook 'emacs-startup-hook
          (lambda ()
            (setq gc-cons-threshold (* 50 1024 1024)
                  gc-cons-percentage 0.1)
            (message "  Emacs loaded in %.2fs (%d GCs)"
                     (float-time
                      (time-subtract after-init-time before-init-time))
                     gcs-done)))

;; ====================================================
;; Package Management (ELPA / MELPA / NONGNU)
;; ====================================================

(require 'package)

(setq package-archives
      '(("melpa"        . "https://melpa.org/packages/")
        ("melpa-stable" . "https://stable.melpa.org/packages/")
        ("gnu"          . "https://elpa.gnu.org/packages/")
        ("nongnu"       . "https://elpa.nongnu.org/nongnu/")))
```

```
(unless package--initialized
  (package-initialize))

(unless package-archive-contents
  (package-refresh-contents))

;; ==================================================
;; use-package bootstrap (safe fallback)
;; ==================================================

(unless (package-installed-p 'use-package)
  (package-install 'use-package))

(require 'use-package)
(setq use-package-always-ensure t)

;; ==================================================
;; Load Path (modular config)
;; ==================================================

(add-to-list 'load-path
             (expand-file-name "lisp" user-emacs-directory))

;; ==================================================
;; Custom file (keep clean init.el)
;; ==================================================

(setq custom-file
      (expand-file-name "custom.el" user-emacs-directory))

(load custom-file 'noerror)

;; ==================================================
;; Core Modules (cross-platform)
;; ==================================================

(require 'init-system)     ;; macOS + Linux system settings
(require 'init-completion) ;; vertico/orderless/marginalia/consult + which-key (before anythi
(require 'init-ui)         ;; theme, fonts, visual setup
(require 'init-editing)    ;; editing behavior
(require 'init-org)        ;; Org mode
(require 'init-roam)       ;; Org-roam
```

```
(require 'init-citations)  ;; citar: .bib citation completion/insertion (after init-roam: use
(require 'init-pdf)        ;; pdf-tools + org-noter (after init-roam: uses org-roam-directory
(require 'init-latex)      ;; LaTeX + latexmk + AUCTeX
(require 'init-company)    ;; completion system
(require 'init-pandoc)     ;; pandoc to convert docx

;; ==================================================
;; UI cleanup (optional global defaults)
;; ==================================================

(setq inhibit-startup-screen nil
      inhibit-startup-message nil)

(column-number-mode 1)
(save-place-mode 1)

;; ==================================================
;; Startup summary
;; ==================================================

(add-hook 'emacs-startup-hook
          (lambda ()
            (message "  Academic Emacs ready: Org-roam + LaTeX + Citations + PDF")))

(provide 'init)

;;; init.el ends here
```

3 init-ui.el

- Place the init-ui.el file in ~/.emacs.d/lisp/

```
;;; init-ui.el --- Visual appearance and fonts

(use-package doom-themes
  :ensure t
  :config
  (condition-case nil
      (load-theme 'doom-one t)
    (error
     (progn
       (message "doom-one not found, falling back to wombat theme.")
       (load-theme 'wombat t)))))

(defun my/setup-font-based-on-resolution ()
  "Adjust font size automatically based on display resolution, with terminal fallback."
  (interactive)
  (if (display-graphic-p)
      (let* ((display-width (display-pixel-width))
             (font-profiles '((3000 . (:font "Monaco" :size 200))
                              (2500 . (:font "Monaco" :size 190))
                              (1920 . (:font "Monaco" :size 180))
                              (0    . (:font "Monaco" :size 170))))
             (selected-profile
              (cl-loop for (min-width . settings) in font-profiles
                       when (> display-width min-width)
                       return settings)))
        (when selected-profile
          (let ((font-name (plist-get selected-profile :font))
                (font-size (plist-get selected-profile :size)))
            (when (member font-name (font-family-list))
              (set-face-attribute 'default nil :font font-name :height font-size)
              (message "GUI: Font set to %s (%d)" font-name font-size)))))
    ;; fallback for terminal mode
    (set-face-attribute 'default nil :height 130)
```

```
    (message "TTY: Default font height set to 130")))

(add-hook 'window-setup-hook #'my/setup-font-based-on-resolution)

(global-display-line-numbers-mode 1)
(global-visual-line-mode 1)

(provide 'init-ui)
;;; init-ui.el ends here
```

4 init-editing.el

- Place the init-editing.el file in ~/.emacs.d/lisp/

```
;;; init-editing.el --- General editing enhancements

(use-package smex
  :bind ("M-x" . smex))

(use-package smartparens
  :config
  (smartparens-global-mode t)
  (show-smartparens-global-mode t)
  (sp-pair "\\[" "\\]"))

(use-package rainbow-delimiters
  :hook (prog-mode . rainbow-delimiters-mode))

(use-package rainbow-identifiers
  :hook ((prog-mode latex-mode LaTeX-mode) . rainbow-identifiers-mode))

(use-package adaptive-wrap
  :hook (visual-line-mode . adaptive-wrap-prefix-mode)
  :config
  (setq-default adaptive-wrap-extra-indent 0))

(use-package visual-fill-column
  :config
  (setq-default fill-column 99999))

;; Spell checking
(setq ispell-program-name "aspell")
(setq ispell-dictionary "en")

(dolist (hook '(text-mode-hook org-mode-hook))
  (add-hook hook 'flyspell-mode))
(add-hook 'prog-mode-hook 'flyspell-prog-mode)
```

```
(global-set-key (kbd "C-;") 'flyspell-correct-wrapper)

(use-package flyspell
  :ensure t
  :hook ((text-mode . flyspell-mode)
         (org-mode  . flyspell-mode)
         (prog-mode . flyspell-prog-mode))
  :config
  (setq ispell-program-name "aspell"
        ispell-dictionary "en"))

(use-package flyspell-correct
  :ensure t
  :after flyspell
  :bind (:map flyspell-mode-map
              ("C-;" . flyspell-correct-wrapper)))

;; Custom handy shortcut
(setq debug-on-error t)
(electric-indent-mode -1)

(global-set-key (kbd "C-x w")
  (lambda ()
    (interactive)
    (save-excursion
      (forward-char)
      (backward-sexp)
      (let ((pos (point)))
        (forward-sexp)
        (kill-ring-save pos (point))))))

(provide 'init-editing)
;;; init-editing.el ends here
```

5 init-org.el

- Place the init-org.el file in ~/.emacs.d/lisp/

```
;;; init-org.el --- Org mode configuration

(use-package org
  :ensure t
  :hook ((org-mode . org-indent-mode)
         (org-mode . visual-line-mode))
  :config
  (setq org-log-done 'time
        org-startup-folded t
        org-icalendar-include-todo t
        org-hide-emphasis-markers t
        org-pretty-entities t
        org-return-follows-link t))

;; Optional: modern org appearance
(use-package org-modern
  :ensure t
  :hook (org-mode . org-modern-mode))

(provide 'init-org)
;;; init-org.el ends here
```

6 init-roam.el

- Place the init-roam.el file in ~/.emacs.d/lisp/

```
;;; init-roam.el --- Org-roam setup

(use-package org-roam
  :ensure t
  :demand t  ;; Ensure org-roam is loaded by default
  :init
  (setq org-roam-v2-ack t)
  :custom
  (org-roam-directory "~/MEGA/org/roam")
  (org-roam-completion-everywhere t)
  :bind (("C-c n l" . org-roam-buffer-toggle)
         ("C-c n f" . org-roam-node-find)
         ("C-c n i" . org-roam-node-insert)
         ("C-c n I" . org-roam-node-insert-immediate)
         ("C-c n p" . my/org-roam-find-project)
         ("C-c n t" . my/org-roam-capture-task)
         ("C-c n b" . my/org-roam-capture-inbox)
         :map org-mode-map
         ("C-M-i" . completion-at-point)
         :map org-roam-dailies-map
         ("Y" . org-roam-dailies-capture-yesterday)
         ("T" . org-roam-dailies-capture-tomorrow))
  :bind-keymap
  ("C-c n d" . org-roam-dailies-map)
  :config
  (require 'org-roam-dailies) ;; Ensure the keymap is available
  (org-roam-db-autosync-mode))

(defun org-roam-node-insert-immediate (arg &rest args)
  (interactive "P")
  (let ((args (push arg args))
        (org-roam-capture-templates (list (append (car org-roam-capture-templates)
                                                  '(:immediate-finish t))))))
```

```
    (apply #'org-roam-node-insert args)))

(defun my/org-roam-filter-by-tag (tag-name)
  (lambda (node)
    (member tag-name (org-roam-node-tags node))))

(defun my/org-roam-list-notes-by-tag (tag-name)
  (mapcar #'org-roam-node-file
          (seq-filter
           (my/org-roam-filter-by-tag tag-name)
           (org-roam-node-list))))

(defun my/org-roam-refresh-agenda-list ()
  (interactive)
  (setq org-agenda-files (my/org-roam-list-notes-by-tag "Project")))

;; Build the agenda list the first time for the session
(my/org-roam-refresh-agenda-list)

(defun my/org-roam-project-finalize-hook ()
  "Adds the captured project file to `org-agenda-files' if the
capture was not aborted."
  ;; Remove the hook since it was added temporarily
  (remove-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Add project file to the agenda list if the capture was confirmed
  (unless org-note-abort
    (with-current-buffer (org-capture-get :buffer)
      (add-to-list 'org-agenda-files (buffer-file-name)))))

(defun my/org-roam-find-project ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Select a project file to open, creating it if necessary
  (org-roam-node-find
   nil
   nil
   (my/org-roam-filter-by-tag "Project")
   :templates
   '(("p" "project" plain "* Goals\n\n%?\n\n* Tasks\n\n** TODO Add initial tasks\n\n* Dates\n
```

```
      :if-new (file+head "%<%Y%m%d%H%M%S>-${slug}.org" "#+title: ${title}\n#+category: ${tit
      :unnarrowed t))))

(defun my/org-roam-capture-inbox ()
  (interactive)
  (org-roam-capture- :node (org-roam-node-create)
                     :templates '(("i" "inbox" plain "* %?"
                                   :if-new (file+head "Inbox.org" "#+title: Inbox\n")))))

(defun my/org-roam-capture-task ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Capture the new task, creating the project file if necessary
  (org-roam-capture- :node (org-roam-node-read
                            nil
                            (my/org-roam-filter-by-tag "Project"))
                     :templates '(("p" "project" plain "** TODO %?"
                                   :if-new (file+head+olp "%<%Y%m%d%H%M%S>-${slug}.org"
                                                          "#+title: ${title}\n#+category: ${
                                                          ("Tasks"))))))

(defun my/org-roam-copy-todo-to-today ()
  (interactive)
  (let ((org-refile-keep t) ;; Set this to nil to delete the original!
        (org-roam-dailies-capture-templates
          '(("t" "tasks" entry "%?"
             :if-new (file+head+olp "%<%Y-%m-%d>.org" "#+title: %<%Y-%m-%d>\n" ("Tasks")))))
        (org-after-refile-insert-hook #'save-buffer)
        today-file
        pos)
    (save-window-excursion
      (org-roam-dailies--capture (current-time) t)
      (setq today-file (buffer-file-name))
      (setq pos (point)))

    ;; Only refile if the target file is different than the current file
    (unless (equal (file-truename today-file)
                   (file-truename (buffer-file-name)))
      (org-refile nil nil (list "Tasks" today-file nil pos)))))
```

```
(add-to-list 'org-after-todo-state-change-hook
             (lambda ()
               (when (equal org-state "DONE")
                 (my/org-roam-copy-todo-to-today))))

(provide 'init-roam)
;;; init-roam.el ends here
```

7 init-latex.el

- Place the init-latex.el file in ~/.emacs.d/lisp/

```
;;; init-latex.el --- LaTeX/AUCTeX + latexmk configuration

(use-package tex
  :ensure auctex
  :defer t
  :hook (LaTeX-mode . my/latex-setup)
  :config
  (setq TeX-auto-save t
        TeX-parse-self t
        TeX-save-query nil
        TeX-PDF-mode t
        TeX-source-correlate-mode t
        TeX-source-correlate-method 'synctex)

  (setq TeX-view-program-selection
        '(((output-pdf "PDF Viewer"))))

  ;; Use Skim with SyncTeX or fallback to Preview.app
  (setq TeX-view-program-list
        '(("PDF Viewer" "/Applications/Skim.app/Contents/SharedSupport/displayline -b -g %n %

  (add-to-list 'TeX-command-list
               '("LatexMk"
                 "latexmk -pdf -synctex=1 -interaction=nonstopmode %s"
                 TeX-run-TeX nil t
                 :help "Run latexmk for PDF output"))

  (setq TeX-command-default "LatexMk"))

(defun my/latex-setup ()
  "My custom LaTeX setup."
  (turn-on-reftex)
  (setq reftex-plug-into-AUCTeX t)
```

```
  (flyspell-mode 1)
  (LaTeX-math-mode 1)
  (visual-line-mode 1))

(provide 'init-latex)
;;; init-latex.el ends here
```

8 init-company.el

- Place the init-company.el file in ~/.emacs.d/lisp/

```
;;; init-company.el --- Company-mode configuration

(use-package company
  :hook ((after-init . global-company-mode)
         (LaTeX-mode . company-mode))
  :config
  (setq company-idle-delay 0.2
        company-minimum-prefix-length 2
        company-tooltip-align-annotations t
        company-dabbrev-downcase nil))

(provide 'init-company)
;;; init-company.el ends here
```

9 init-os.el

- Place the init-os.el file in ~/.emacs.d/lisp/

```
;; ----------------------------------------
;;   MacOS-Specific Setup for Emacs
;; ----------------------------------------

;;   Startup: Performance tuning
(setq native-comp-async-report-warnings-errors nil) ; silence native-comp warnings
(setq gc-cons-threshold (* 50 1000 1000))           ; increase GC threshold for faster startu

(add-hook 'emacs-startup-hook
          (lambda ()
            (message "  Emacs ready in %.2f seconds with %d GCs."
                     (float-time (time-subtract after-init-time before-init-time))
                     gcs-done)))

;;   Environment: Mac shell paths
(use-package exec-path-from-shell
  :if (memq window-system '(mac ns x))
  :config
  (exec-path-from-shell-initialize))

;; Add MacPorts paths for TeXLive and Hunspell
(let ((my-paths '("/opt/local/libexec/texlive/texbin"
                  "/opt/local/bin")))
  (setenv "PATH" (concat (mapconcat #'identity my-paths ":") ":" (getenv "PATH")))
  (setq exec-path (append my-paths exec-path)))

;;   Visuals: Theme, font, retina, fullscreen
(when (eq system-type 'darwin)
  ;; Prefer dark mode and unified title bar
  (add-to-list 'default-frame-alist '(ns-appearance . dark))
  (add-to-list 'default-frame-alist '(ns-transparent-titlebar . t)))

(setq frame-resize-pixelwise t)                          ; Retina/HiDPI fix
```

```
(add-to-list 'default-frame-alist '(fullscreen . maximized)) ; Start maximized
(set-fontset-font t 'symbol (font-spec :family "Apple Color Emoji") nil 'prepend)

;;  Mouse & Trackpad: Smooth scrolling
(setq mouse-wheel-scroll-amount '(1 ((shift) . 1))) ; 1 line at a time
(setq mouse-wheel-progressive-speed nil)
(setq scroll-step 1)

;;  Clipboard and Input
(setq select-enable-clipboard t)
(setq save-interprogram-paste-before-kill t)
(setq use-dialog-box nil) ; no popups

;; Optional: Better Japanese keyboard handling (¥ = \)
;; (define-key global-map [?¥] [?\\])

;;  UI Hygiene (uncomment if desired)
;; (menu-bar-mode -1)
;; (tool-bar-mode -1)
;; (scroll-bar-mode -1)

(provide 'init-macos)
```

10 init-pandoc.el

```
;;; init-pandoc.el --- Pandoc integration -*- lexical-binding: t; -*-

;;; Commentary:
;; Cross-platform Pandoc workflow for:
;; - macOS
;; - Ubuntu/Linux
;; - LaTeX → DOCX
;; - Quarto projects
;; - BibTeX citations
;; - Word templates

;;; Code:

(require 'cl-lib)

(defgroup my-pandoc nil
  "Pandoc workflow utilities."
  :group 'tools)

;; ------------------------------------------------
;; Locate Pandoc
;; ------------------------------------------------

(defcustom my-pandoc-command
  (or (executable-find "pandoc")
      "/usr/bin/pandoc")
  "Pandoc executable."
  :type 'string)

(defcustom my-pandoc-default-reference-doc nil
  "Fallback Word template."
  :type '(choice file (const nil)))

;; ------------------------------------------------
```

```
;; Helpers
;; ------------------------------------------------

(defun my-pandoc--project-root ()
  "Return project root."
  (or (when (fboundp 'project-current)
        (when-let ((proj (project-current nil)))
          (project-root proj)))
      default-directory))

(defun my-pandoc--find-bib ()
  "Find bibliography file."
  (let ((root (my-pandoc--project-root)))
    (car (append
          (directory-files root t "\\.bib$")
          nil))))

(defun my-pandoc--find-reference-doc ()
  "Find reference.docx."
  (let ((root (my-pandoc--project-root)))
    (or (car (directory-files root t "reference\\.docx$"))
        my-pandoc-default-reference-doc)))

(defun my-pandoc--docx-name (texfile)
  "Generate output DOCX filename."
  (concat
   (file-name-sans-extension texfile)
   ".docx"))

(defun my-pandoc--open-file (file)
  "Open FILE on current OS."

  (cond
   ((eq system-type 'darwin)
    (start-process "pandoc-open" nil "open" file))

   ((eq system-type 'gnu/linux)
    (start-process "pandoc-open" nil "xdg-open" file))

   ((eq system-type 'windows-nt)
    (w32-shell-execute "open" file))))
```

```
;; ------------------------------------------------
;; Async Runner
;; ------------------------------------------------

(defun my-pandoc--run (args output)
  "Run Pandoc asynchronously."

  (let ((buffer (get-buffer-create "*Pandoc*")))

    (with-current-buffer buffer
      (erase-buffer))

    (make-process
     :name "pandoc"
     :buffer buffer
     :command (cons my-pandoc-command args)
     :sentinel
     (lambda (proc event)
       (when (string-match-p "finished" event)
         (message "Pandoc finished: %s" output))))

    (display-buffer buffer)))

;; ------------------------------------------------
;; Basic Conversion
;; ------------------------------------------------

(defun my/pandoc-tex-to-docx ()
  "Convert current TeX file to DOCX."

  (interactive)

  (unless buffer-file-name
    (user-error "Buffer not visiting a file"))

  (save-buffer)

  (let* ((texfile buffer-file-name)
         (output (my-pandoc--docx-name texfile)))

    (my-pandoc--run
     (list texfile
```

```
           "-o"
           output)
     output)))

;; ------------------------------------------------
;; Citations
;; ------------------------------------------------

(defun my/pandoc-tex-to-docx-citeproc ()
  "Convert TeX using BibTeX."

  (interactive)

  (save-buffer)

  (let* ((texfile buffer-file-name)
         (bibfile (my-pandoc--find-bib))
         (output (my-pandoc--docx-name texfile)))

    (unless bibfile
      (user-error "No .bib file found"))

    (my-pandoc--run
     (list texfile
           "--citeproc"
           (concat "--bibliography=" bibfile)
           "-o"
           output)
     output)))

;; ------------------------------------------------
;; Journal Version
;; ------------------------------------------------

(defun my/pandoc-tex-to-docx-journal ()
  "Convert using citations and Word template."

  (interactive)

  (save-buffer)

  (let* ((texfile buffer-file-name)
```

```
         (bibfile (my-pandoc--find-bib))
         (refdoc (my-pandoc--find-reference-doc))
         (output (my-pandoc--docx-name texfile))
         (args (list texfile
                     "--citeproc"
                     "--number-sections")))

    (when bibfile
      (setq args
            (append args
                    (list
                     (concat "--bibliography=" bibfile)))))

    (when refdoc
      (setq args
            (append args
                    (list
                     (concat "--reference-doc=" refdoc)))))

    (setq args
          (append args
                  (list "-o" output)))

    (my-pandoc--run args output)))

;; ------------------------------------------------
;; Open Generated DOCX
;; ------------------------------------------------

(defun my/pandoc-open-docx ()
  "Open DOCX associated with current TeX file."

  (interactive)

  (let ((docx
         (my-pandoc--docx-name
          buffer-file-name)))

    (if (file-exists-p docx)
        (my-pandoc--open-file docx)
      (message "No DOCX found"))))
```

```
;; ------------------------------------------------
;; Convenience Menu
;; ------------------------------------------------

(defun my/pandoc-menu ()
  "Pandoc command menu."

  (interactive)

  (pcase
      (completing-read
       "Pandoc: "
       '("Basic DOCX"
         "DOCX + Citations"
         "Journal DOCX"
         "Open DOCX"))
    ("Basic DOCX"
     (my/pandoc-tex-to-docx))
    ("DOCX + Citations"
     (my/pandoc-tex-to-docx-citeproc))
    ("Journal DOCX"
     (my/pandoc-tex-to-docx-journal))
    ("Open DOCX"
     (my/pandoc-open-docx))))

;; ------------------------------------------------
;; Keybindings
;; ------------------------------------------------

(global-set-key (kbd "C-c p p") #'my/pandoc-menu)

(global-set-key (kbd "C-c p d")
                #'my/pandoc-tex-to-docx)

(global-set-key (kbd "C-c p c")
                #'my/pandoc-tex-to-docx-citeproc)

(global-set-key (kbd "C-c p j")
                #'my/pandoc-tex-to-docx-journal)

(global-set-key (kbd "C-c p o")
                #'my/pandoc-open-docx)
```

```
(provide 'init-pandoc)

;;; init-pandoc.el ends here
```

10.0.1 With this layout:

- C-c p d → simple Word export
- C-c p c → Word export with bibliography
- C-c p j → journal-style Word export with bibliography + template
- C-c p o → open the generated .docx
- C-c p p → interactive menu

11 User guide

A working reference for your Emacs setup: writing LaTeX papers, taking notes in Org/org-roam, citing from `.bib` files, and reading/annotating PDFs — the same way on macOS and Ubuntu.

11.1 1. How it's organized

```
~/.emacs.d/
   early-init.el           # GC tuning + package-system setup (loads before everything)  [NEW
   init.el                 # bootstraps package.el/use-package, requires everything below
   lisp/
       init-system.el      # OS settings + shared resolution helpers (loads first)
       init-completion.el   # vertico/orderless/marginalia/consult + which-
key            [NEW]
       init-ui.el           # theme, fonts, visual behavior
       init-editing.el      # smartparens, spell-check, wrapping, misc editing
       init-org.el          # Org mode + org-cite export config
       init-roam.el         # org-roam: notes, dailies, capture templates
       init-citations.el    # citar: .bib-based citation completion       [NEW]
       init-pdf.el           # pdf-tools + org-noter: read/annotate PDFs   [NEW]
       init-latex.el        # AUCTeX + latexmk + pdf-tools preview
       init-company.el      # in-buffer completion (company-mode)
       init-pandoc.el       # LaTeX → DOCX export for journal submission
```

Load order matters. `early-init.el` runs before `package.el` and before the frame is created — it's where GC tuning and `package-enable-at-startup` belong, since setting them later has little effect. `init-system.el` loads first among the `lisp/` modules because it defines helper functions (`my/first-existing-directory`, `my/first-existing-file`, `my/first-executable`) that later modules use to resolve paths and tools at runtime instead of hardcoding one machine's layout. `init-completion.el` loads right after it, before anything that uses `completing-read` (citar, org-roam). `init-citations.el` and `init-pdf.el` load after `init-roam.el` because they both reference `org-roam-directory`.

11.2 2. What's new, and why it's worth using

11.2.1 Citations — `init-citations.el` (citar)

Before, there was no way to insert a citation while writing — you'd alt-tab to look up a key or type it from memory. Now:

Command	Where	What it does
`C-c b b`	Org or LaTeX buffer	Search your `.bib` file and insert a citation at point
`C-c b o`	Anywhere	Open the PDF/file attached to a reference (if you have one linked)
`C-c b r`	Anywhere	Open your notes on a reference
`C-c b i`	Org buffer	`org-cite-insert` (the native Org citation UI, if you prefer it to citar's)

It works identically whether you're in a `.org` file (via `org-cite`, Org's built-in citation system since 9.5) or a `.tex` file (via AUCTeX). One `.bib` file, one keybinding, both formats.

11.2.2 PDF reading and annotation — `init-pdf.el` (pdf-tools + org-noter)

`init-ui.el` already had a hook disabling line numbers in `pdf-view-mode` — implying PDF viewing was planned but the package itself was never wired in. Now it is:

- **pdf-tools** replaces the built-in DocView. Real text selection, real search, annotations, much faster rendering.
- **org-noter** lets you open a PDF and an Org notes buffer side by side — your notes stay synced to the page you're reading, and they live in your org-roam directory, so literature notes are searchable alongside everything else.
- Installation is deferred (`pdf-loader-install`) until you actually open a PDF, so it doesn't slow down startup.

11.2.3 LaTeX preview via pdf-tools, not a separate app

`init-latex.el` used to branch on OS: Skim on macOS, Evince on Ubuntu, both hardcoded paths. It now uses pdf-tools itself as the preview target (`TeX-view-program-selection '((output-pdf "PDF Tools"))`), so:

- Compiling with `C-c C-c` → `LatexMk` opens/updates the PDF **inside Emacs**, in the same viewer you use for reading papers.
- SyncTeX (`C-c C-v` for forward search, click-to-jump for inverse search) works through pdf-tools instead of an external app's own SyncTeX support.
- No dependency on Skim or Evince being installed at all, on either OS. (A fallback to a discovered external viewer only kicks in if you're running Emacs without a graphical frame, e.g. over SSH.)

11.2.4 Cross-platform path/tool resolution — `init-system.el`

Three helper functions, used throughout the rest of the config instead of hardcoded single paths:

```
(my/first-existing-directory "~/MEGA/org" "~/Org" "~/org")
(my/first-existing-file "~/MEGA/bibliography/references.bib" ...)
(my/first-executable "aspell" "hunspell" "ispell")
```

Each tries its candidates in order and returns the first one that actually exists on *this* machine. This is what makes the same `init.el` work on both your macOS and Ubuntu installs without editing it per-machine — see §5 if you need to add a new candidate path.

11.2.5 Minibuffer completion & discoverability — `init-completion.el`

The single biggest day-to-day usability upgrade in this config. Before, citar's citation search, `org-roam-node-find`, and `M-x` all went through Emacs' plain default minibuffer — exact prefix matching, no previews, no annotations.

- **vertico** — vertical, incremental completion UI for every minibuffer prompt.
- **orderless** — match candidates in any word order (e.g. typing `smith 2020` finds "Smith, J. (2020) …" in a citation search regardless of which word you typed first). This is what makes searching a large `.bib` file or a roam vault actually fast.
- **marginalia** — rich inline annotations (docstrings next to `M-x` commands, file sizes/dates next to `find-file` candidates).
- **consult** — a set of drop-in replacement commands with live preview (buffer switching, in-buffer search, jumping to headings).
- **which-key** — pops up available keybindings automatically when you pause after a prefix key like `C-c b` or `C-c n`, so you don't need to keep this handbook open to remember them.

11.2.6 Faster, cleaner startup — `early-init.el`

Moved `gc-cons-threshold` and `package-enable-at-startup` out of `init.el` and into a new `early-init.el`, which Emacs (27+) loads before `package.el` runs its own automatic setup and before the frame is created. Setting `package-enable-at-startup` inside `init.el` (where it lived before) had essentially no effect, since Emacs had already acted on it by the time `init.el` loaded — `early-init.el` is the only place it actually does anything. The GC threshold is set very high during startup here, then brought back down to a sane runtime value (50MB) once `emacs-startup-hook` fires, from `init.el`.

11.3 3. Daily workflow keybindings

11.3.1 Writing (LaTeX)

Key	Action
`C-c C-c`	Compile (runs `latexmk`, default command)
`C-c C-v`	View/preview PDF, jump to current point (forward search)
`C-c b b`	Insert citation (citar)
`C-c ]` / RefTeX menu	Insert cross-reference (`turn-on-reftex` is on by default)

11.3.2 Notes (Org / org-roam)

Key	Action
`C-c n f`	Find or create a roam node
`C-c n i`	Insert a link to a roam node
`C-c n b`	Toggle the roam backlinks buffer
`C-c n t`	Capture a task under a project node
`C-c n p`	Find-or-create a project node
`C-c n I`	Insert a node link and finish capture immediately (no extra prompts)
`C-c n d`	Dailies keymap (e.g. `C-c n d Y` = yesterday's daily note, `T` = tomorrow's)
`C-c b i`	Insert citation via native `org-cite-insert`

11.3.3 Reading PDFs

Key	Action
`j` / `k`	Next/previous line-or-page
`h` / `l`	Previous/next page
`C-c b o`	Open the PDF linked to a reference (citar)
org-noter (once launched on a PDF)	Notes buffer stays synced to your position in the PDF

11.3.4 Navigation & search (consult / which-key)

Key	Action
`C-x b`	`consult-buffer` — switch buffers/recent files/bookmarks with live preview
`C-c s l`	`consult-line` — fuzzy search current buffer
`C-c s r`	`consult-ripgrep` — project-wide search (needs `ripgrep` installed separately)
`C-c s o`	`consult-outline` — jump to a heading/section in the current buffer
`C-c s h`	`consult-org-heading` — jump to any heading in the current Org file
`M-y`	`consult-yank-pop` — browse kill-ring with preview
(pause after any `C-c` prefix)	which-key shows what's available — use this instead of memorizing the tables above

11.3.5 Converting to DOCX for journal submission

Key	Action
`C-c p p`	Pandoc menu (pick basic/citations/journal/open)
`C-c p d`	Basic TeX → DOCX
`C-c p c`	TeX → DOCX with citations (`--citeproc`, auto-finds a `.bib` in the project)
`C-c p j`	Journal version: citations + numbered sections + a `reference.docx` template if one exists in the project folder

Key	Action
`C-c p o`	Open the generated DOCX

11.4 4. Maintenance checklist

Run these occasionally, not just when something breaks:

- `M-x package-install-selected-packages` — reinstalls anything `use-package :ensure t` expects but doesn't currently have. Cheap insurance after an interrupted first-run install (see §6).
- `M-x package-refresh-contents` then `M-x package-list-packages`, `U` then `x` — the normal "update everything" flow. Do this every month or so; AUCTeX, org-roam, and pdf-tools all move.
- **Check `*Messages*` (`C-h e`)** after any Emacs restart following a config change — install/compile failures show up there in red, easy to miss otherwise.
- `EMACS_DEBUG=1 emacs` — turns `debug-on-error` back on for a session, without permanently enabling it (it's off by default in `init-editing.el` now, so ordinary errors don't interrupt writing).

11.5 5. Adding a new machine, or a folder in a different place

If a third machine mounts your notes/bibliography somewhere else, you don't need to change every file — just add the new path to the relevant candidate list:

- **Org notes root**: `my/org-root` in `init-org.el`
- **org-roam notes**: the `org-roam-directory` candidate list in `init-roam.el`
- **Bibliography**: `my-bibliography-files` in `init-citations.el`
- **Spell-checker / pandoc / any other tool**: these already resolve from `PATH` automatically via `my/first-executable`, so a new machine just needs the tool installed, no config change.

Pattern to follow, e.g. adding a new Org root candidate:

```
(defvar my/org-root
  (or (my/first-existing-directory "~/MEGA/org" "~/Org" "~/org" "~/NewPath/org")
      (expand-file-name "org" user-emacs-directory))
  "Root directory for Org notes.")
```

11.6 6. Troubleshooting

Emacs seems frozen on first launch after a config change. Almost always package installation/compilation, not a real hang — `use-package :ensure t` pulls in a lot at once (AUCTeX especially is slow to compile). Give it a few minutes. If you must confirm it's alive rather than stuck: open a terminal, `top`/`htop`, check whether the `emacs` process is using CPU (busy = probably fine) or sitting near 0% (possibly actually blocked, e.g. on network).

Theme looks wrong / lost your colors after an interrupted startup. `init-ui.el` silently falls back to the built-in `wombat` theme if `doom-one` fails to load (e.g. `doom-themes` wasn't fully installed). Check with:

```
M-x package-installed-p RET doom-themes RET
```

If `nil`: `M-x package-refresh-contents`, then `M-x package-install RET doom-themes RET`, then `M-x load-theme RET doom-one RET`.

`package-refresh-contents` or an install hangs specifically. Likely a network/proxy/DNS issue reaching MELPA from that machine, not an Emacs problem. Test from a terminal: `curl -I https://melpa.org/packages/`. If that also hangs or fails, the fix is on the network side, not in `init.el`.

AUCTeX completion (labels/citations/macros) doesn't seem to work in `.tex` files. This was an actual bug we fixed: `init-company.el` used to have the generic completion backends silently override the LaTeX-specific ones due to `add-hook` ordering. If you still see this, confirm you're on the current `init-company.el` (the LaTeX-specific dolist should *not* include `LaTeX-mode-hook`).

A `.bib` file, org-roam folder, or PDF viewer doesn't behave as expected on a given machine. Check what actually resolved: `M-x eval-expression RET my-bibliography-files` or `org-roam-directory` will show you the path that was picked. If it's the fallback rather than what you expected, your synced folder likely isn't mounted where the candidate list expects — see §5.

A wave of `Failed to install X: .../compat-VERSION.tar: Not found` errors, followed by `Cannot load X` for several unrelated packages at once. Not several separate bugs — one shared dependency. `compat` is a small library that vertico, orderless, marginalia, consult, org-appear, citar, and org-noter all rely on. If your local package index still points at a `compat` version that's since been pruned from GNU ELPA, every package that needs it fails at the same download step, and everything downstream cascades into "Cannot load." Fix:

```
M-x package-refresh-contents
M-x package-install RET compat RET
M-x package-install-selected-packages
```

Then restart Emacs. If the refresh doesn't actually pick up a new index (some setups cache more aggressively), force it from a terminal first:

```
rm -f ~/.emacs.d/elpa/archives/gnu/archive-contents
rm -f ~/.emacs.d/elpa/archives/melpa/archive-contents
rm -f ~/.emacs.d/elpa/archives/nongnu/archive-contents
```

then relaunch Emacs and refresh again.

`Cannot load org-noter` (or any single package) with no obvious cause. Before assuming it's specific to that package, check `*Messages*` (`C-h e`) for the *first* error in the sequence, not just the last line — `use-package` often prints a whole chain of failures and the useful one (a missing dependency, a stale archive URL, a network timeout) is usually earlier than the "Cannot load" line itself, which is just the final symptom.

11.7 7. What each module owns (quick lookup)

Module	Owns
`early-init.el`	GC tuning during startup, `package-enable-at-startup`, pre-frame settings
`init-system.el`	OS branching, PATH, `exec-path-from-shell`, resolution helpers
`init-completion.el`	vertico, orderless, marginalia, consult, which-key

Module	Owns
`init-ui.el`	Theme, fonts, emoji, line numbers, scrolling, frame chrome
`init-editing.el`	smartparens, rainbow-delimiters, wrapping, spell-check, `debug-on-error`
`init-org.el`	Org core behavior, TODO states, `org-cite` export processors, `my/org-root`
`init-roam.el`	org-roam directory/capture templates/dailies
`init-citations.el`	citar, `.bib` resolution, citation keybindings
`init-pdf.el`	pdf-tools, org-noter
`init-latex.el`	AUCTeX, latexmk, RefTeX, PDF preview target
`init-company.el`	Completion backends (generic + LaTeX-specific)
`init-pandoc.el`	TeX → DOCX conversion for journal submission